Introduction to Python

🐍 *What is Python?*

Python is a **high-level, easy-to-read programming language** used to build software, websites, automation, data applications.

It is famous for its **simple English-like syntax**, which makes learning and coding very easy.

⭐ *Features of Python*

- ❖ Simple & Easy to Learn
- ❖ Interpreted Language
- ❖ High-level language
- ❖ Platform independent
- ❖ Object-Oriented
- ❖ Dynamic typing(No need to declare data types.)

Used everywhere

Python is not limited to one area. It is used in:

- 🌐 Web development
- 🤖 Artificial Intelligence & Machine Learning
- 📊 Data Science & Analytics
- 🧪 Testing tools

🚀 *Applications of Python*

🌐 Web Development

Python is used to build **backend web applications**.Frameworks: **Django, Flask, FastAPI**

📊 *Data Science & Data Analysis*

Python is the **number 1 language for data work**.

- Libraries: **NumPy, Pandas, Matplotlib**
- Used for: reports, charts, business analysis, *Excel on steroids* 📈

🤖 *Artificial Intelligence & Machine Learning*

Python powers **AI systems**.

- Libraries: **TensorFlow, PyTorch, Scikit-learn**
- Used in: chatbots, recommendation systems, face recognition

---

⚙️ *Automation & Scripting*

Python automates **repetitive tasks**.

- File handling
- Email automation
- System scripts

🧪 *Software Testing*

Used for **test automation**.

- Tools: **Selenium, PyTest**
- Used in: QA, CI/CD pipelines

🖥️ *Desktop Applications*

Python can build **GUI applications**.

- Libraries: **Tkinter, PyQt**
- Used for: calculators, tools, admin apps

🎮 *Game Development*

Python is used for **simple games**.

- Library: **Pygame**
- Good for beginners

🐍 *Python Syntax & Indentation*

Python is famous because its **syntax is simple** and it uses **indentation instead of braces** {}.

1️⃣ *Python Syntax (Simple Rules)*

No semicolons ; Python does **not** need ; at the end of statements

No data type declaration: Python automatically understands the type.

What is Indentation? 🧱

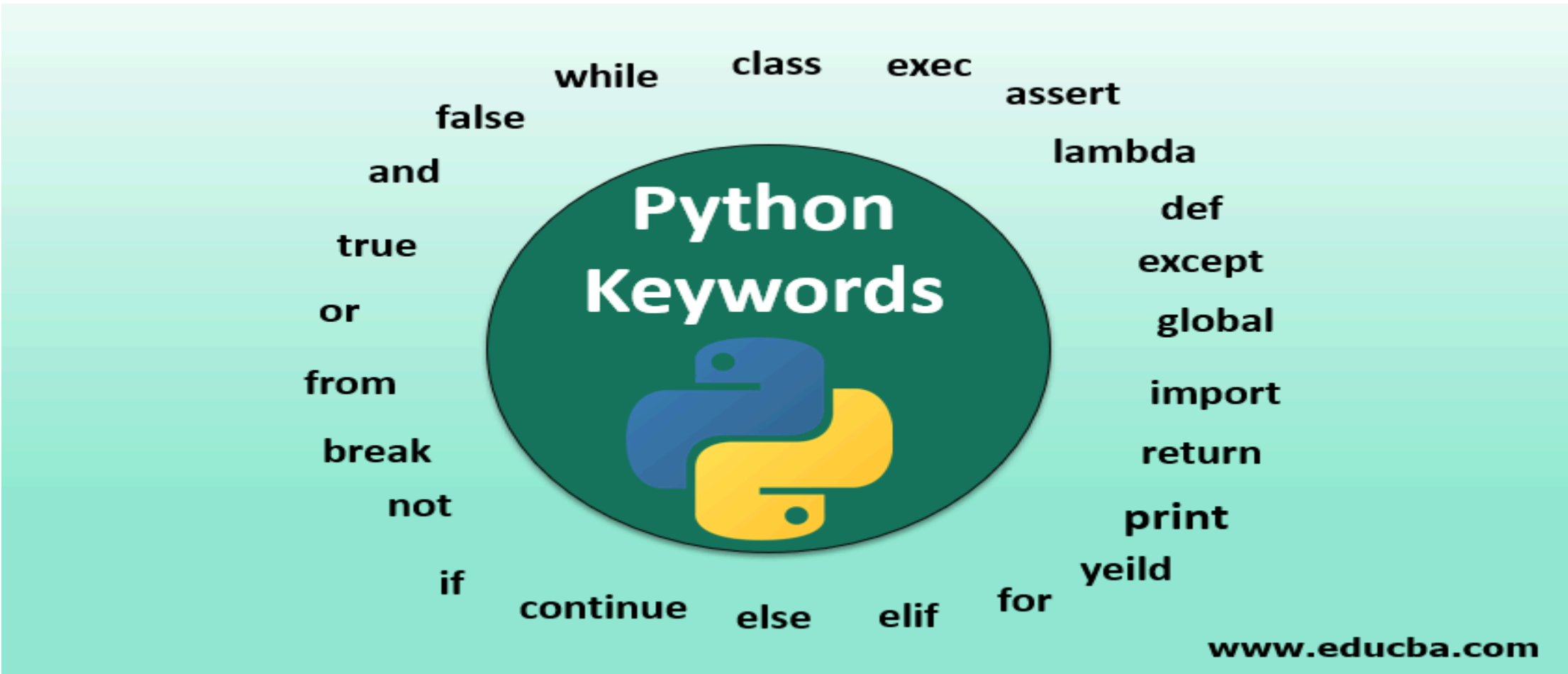
Indentation means spaces at the beginning of a line. In Python, indentation **defines a block of code**. Usually **4 spaces** are used.

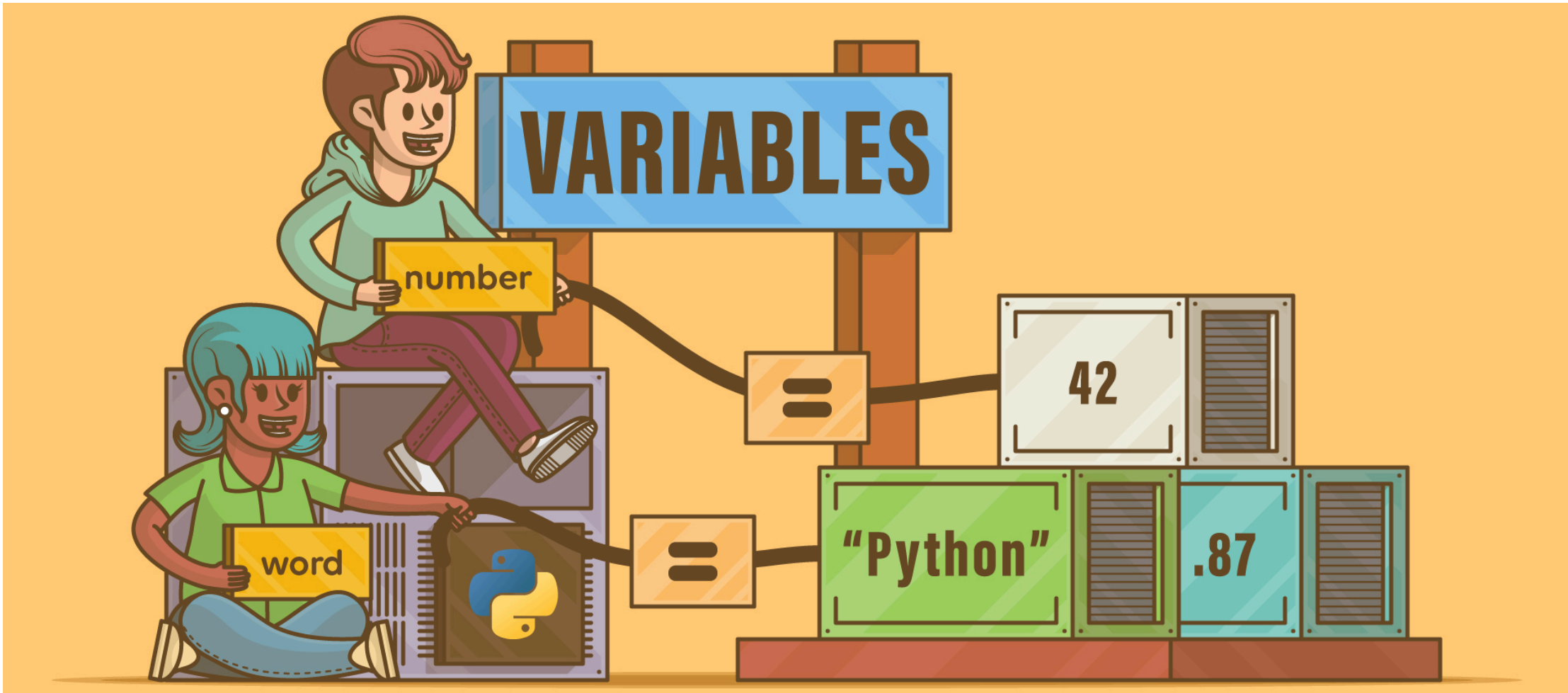
Wrong indentation = program error

- Correct indentation (✅)

```
age = 20
if age > 18:
    print("Adult")
    print("Eligible to vote")
```

🐍 *Python keyword*





- ***What is a Variable?***

A **variable** in Python is a **name used to store data** (value) in memory.
The value can **change during program execution**.

How to create a variable in Python:

No data type needed

```
age = 25

name = "Mohan"

salary = 45000.50
```

Python automatically decides the data type.

---

2 *Dynamic typing (important feature)*

You can change the value and type anytime.

```
x = 10

x = "Python"

x = 3.14
```

✔️ This is allowed in Python.

---

3 *Rules for naming variables*

- Must start with a letter or _
- Can contain letters, digits, _
- ❌ Cannot start with a digit
- ❌ Cannot use keywords

💬 *Comments in Python*

A **comment** is a **note written inside the program** to explain code.
Python **ignores comments** while executing the program.

1️⃣ *Single-line Comment*

```
# This is a single-line comment

print("Hello Python")
```

2️⃣ *Multi-line Comment: Method 2: Triple quotes (''' or """)*

```
"""

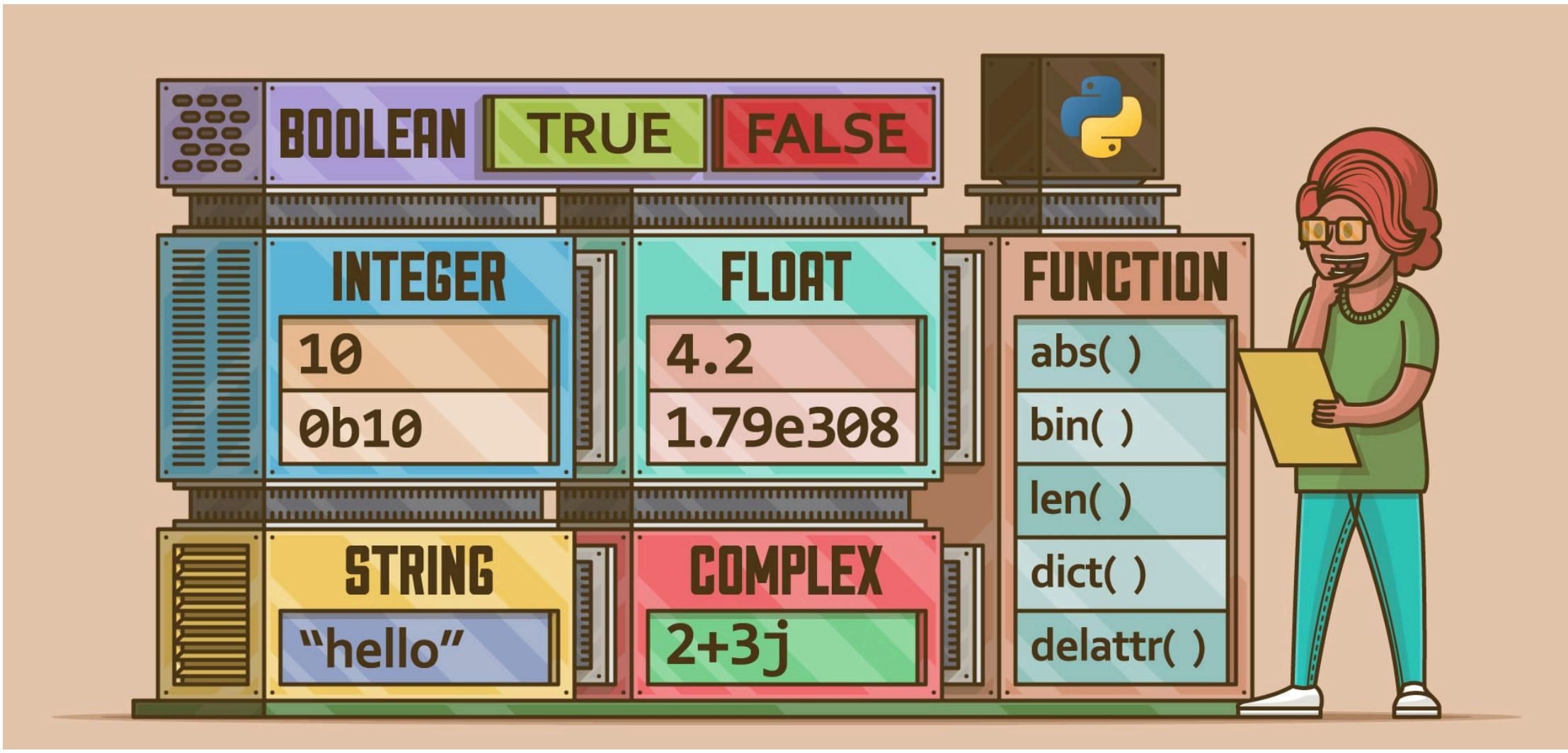
This is a multi-line comment

used for explanation

"""

print("Python")
```

🐍 Data Types in Python



🐍 Data Types in Python

A **data type** tells Python **what kind of value** a variable holds (number, text, true/false, collection, etc.).

1️⃣ Numeric Data Types 🔢

➤ ***int (Integer):*** Whole numbers

```
age = 25
count = -10
```

➤ ***float(Decimal):*** Decimal numbers

```
price = 99.50
pi = 3.14
```

2️⃣ Text Data Type 📝

➤ ***str (String):*** Text or characters

```
name = "Python"

message = 'Hello'
```

3️⃣ *Boolean Data Type*

➤ **bool:** *True or False*

```
isActive = True

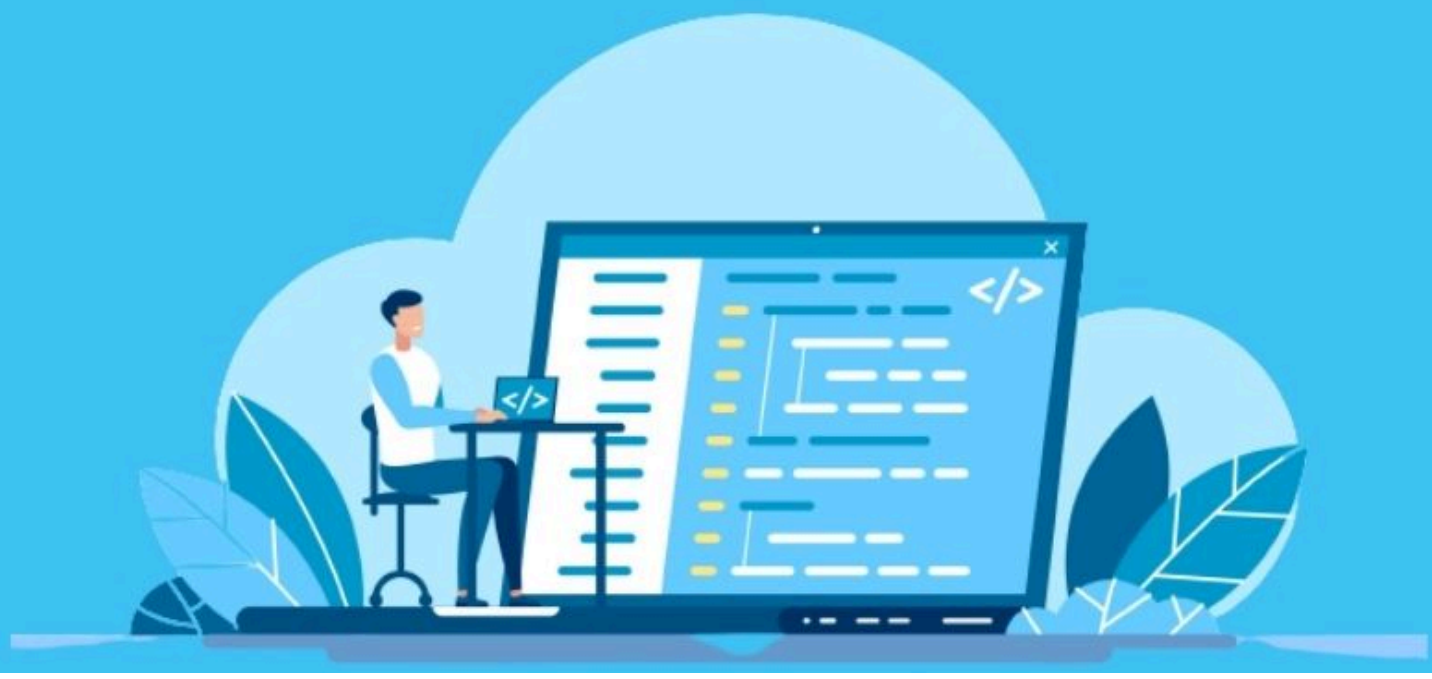
isLoggedIn = False
```

Example 1: Basic Data Types

```
a = 10
b = 3.5
c = "Python"
d = True

print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

Type Casting in Python



- ***What is Type Conversion?***

Type conversion means **changing one data type into another**
for example:

- string → integer,
- integer → float).

Python supports **two types** of conversion

1️⃣ *Implicit Type Conversion (Automatic)*

Done **automatically by Python.** No data loss

```
a = 10        # int
b = 2.5       # float
c = a + b
print(c)          # 12.5
print(type(c))    # float
```

---

2 *Explicit Type Conversion (Type Casting)*

Done **manually by a programmer.** Using built-in functions

➤ *int()*

```
x = "10"

y = int(x)

print(y) # output:10
```

➤ *float()*

```
a = 5

b = float(a)

print(b) #output: 5.0
```

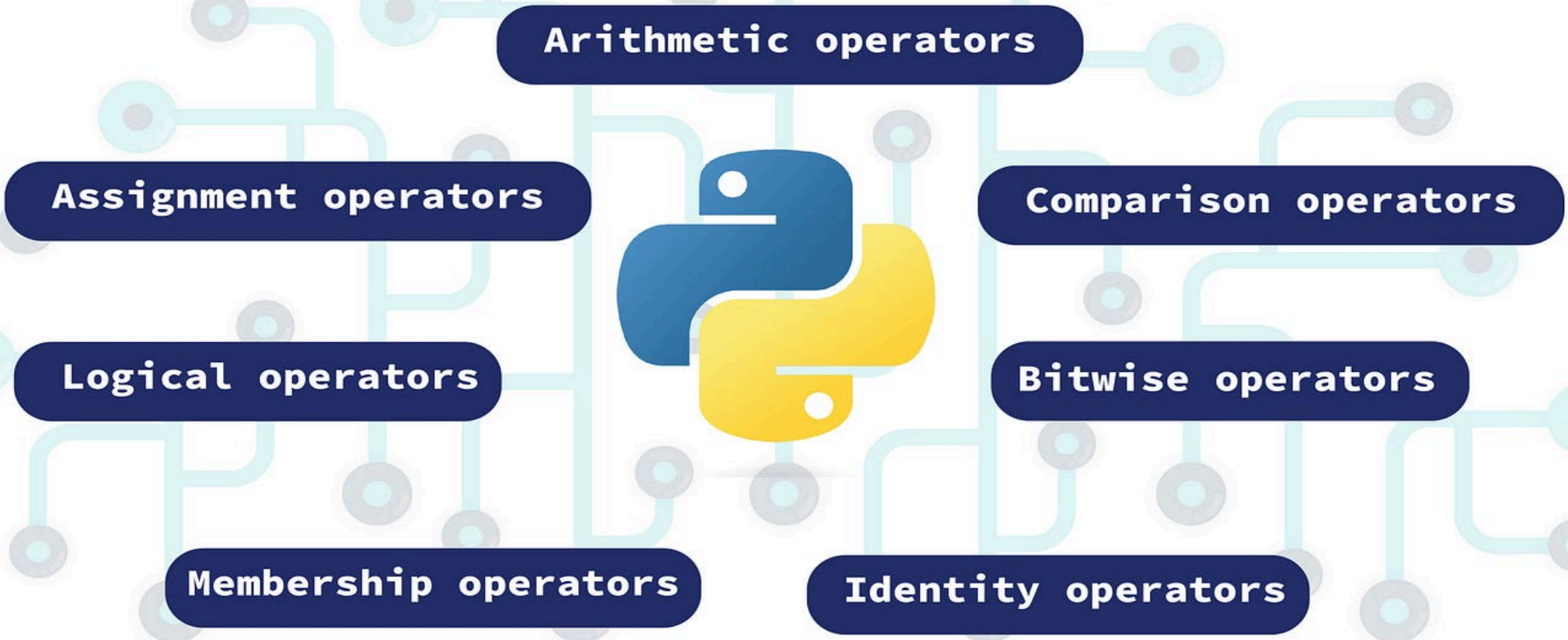
➤ *str()*

```
num = 100
text = str(num)
print(text)#output: 100
```

➤ *bool()*

```
print(bool(0))       # False
print(bool(1))       # True
```

Python Operators



➕ *Arithmetic Operators in Python*

- ***What are Arithmetic Operators?***

Arithmetic operators are used to perform **mathematical calculations** like addition, subtraction, multiplication, etc.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
–	Subtraction	10 – 4	6
*	Multiplication	6 * 2	12
/	Division	8 / 2	4.0
//	Floor Division	9 // 2	4
%	Modulus (Remainder)	10 % 3	1
**	Exponentiation (Power)	2 ** 3	8

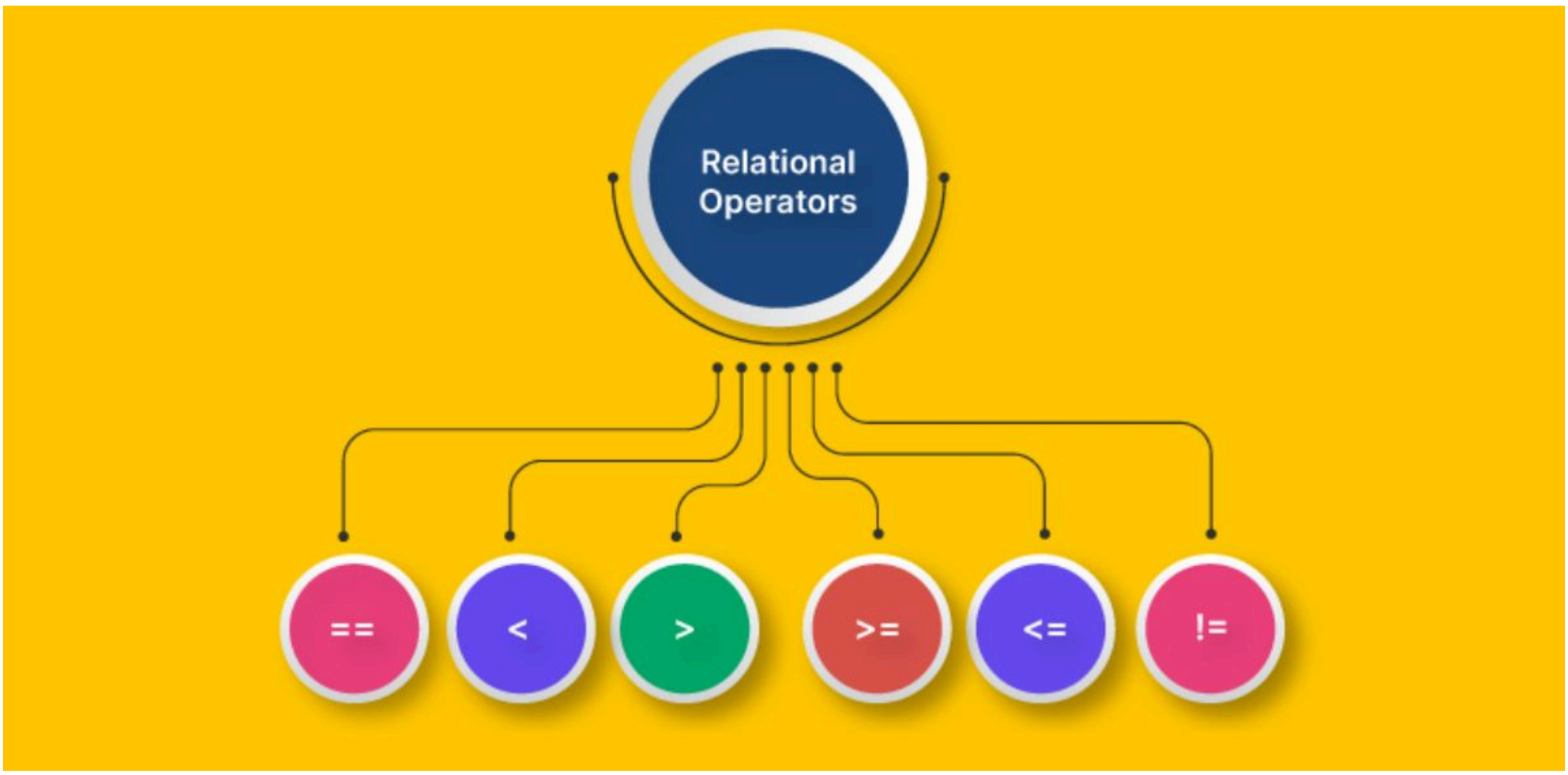
👨‍💻 *Code Example (Python)*

```
# Arithmetic operators example
a = 10
b = 3
print(a + b)    # Addition
print(a - b)    # Subtraction
print(a * b)    # Multiplication
print(a / b)    # Division
print(a % b)    # Modulus
print(a ** b)   # Power
print(a // b)   # Floor division (round down)
```

🔍 *Relational Operators in Python*

- ***What are Relational Operators?***

Relational operators are used to **compare two values**. They always return a **Boolean result** → True or False.



👨‍💻 *Code Example (Python)*

```
# Relational operators example

a = 10

b = 20

print(a > b)     # Greater than

print(a < b)     # Less than

print(a == b)    # Equal to

print(a != b)    # Not equal to

print(a >= b)    # Greater than or equal to

print(a <= b)    # Less than or equal to
```

🧠 *Logical Operators in Python*

🔹 *What are Logical Operators?*

Logical operators are used to **combine multiple conditions** and return a **Boolean result** (True or False).

2️⃣ *Examples of Logical Operators*

➤ AND operator

```
#✔️True only if both conditions are true.

age = 25

has_id = True

print(age > 18 and has_id)
```

➤ OR operator

```
#✔️ True if any one condition is true.

marks = 32

print(marks >= 35 or marks >= 30)
```

➤ NOT operator

```
#✔️ Reverses the result.

isLoggedIn = False

print(not isLoggedIn)
```

👨‍💻 *Code Example (Python)*

```
# Relational operators example

a = 10

b = 20

c = 20

print(a==b and a==b) # false

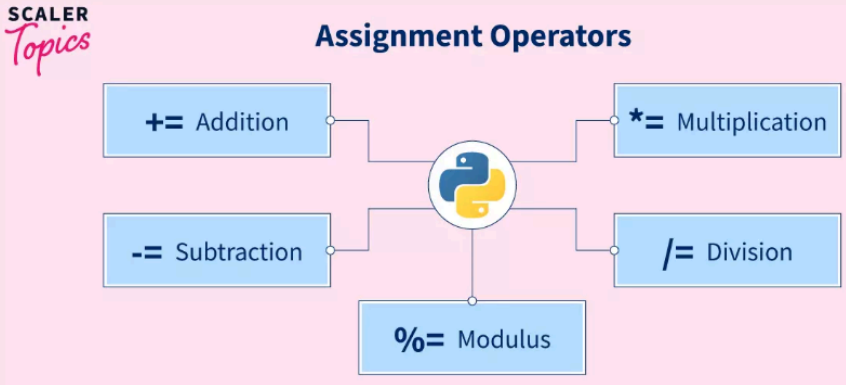
print(a==b or a==b  or b==c)# True

print(a!=b)# true
```

📝 *Assignment Operators in Python*

🔹 *What are Assignment Operators?*

Assignment operators are used to **assign values to variables,** and **update the value** using arithmetic operations.



👨‍💻 *Code Example (Python)*

```python
# Assignment operators example

a = 10       # Assign value

a += 5       # Add and assign

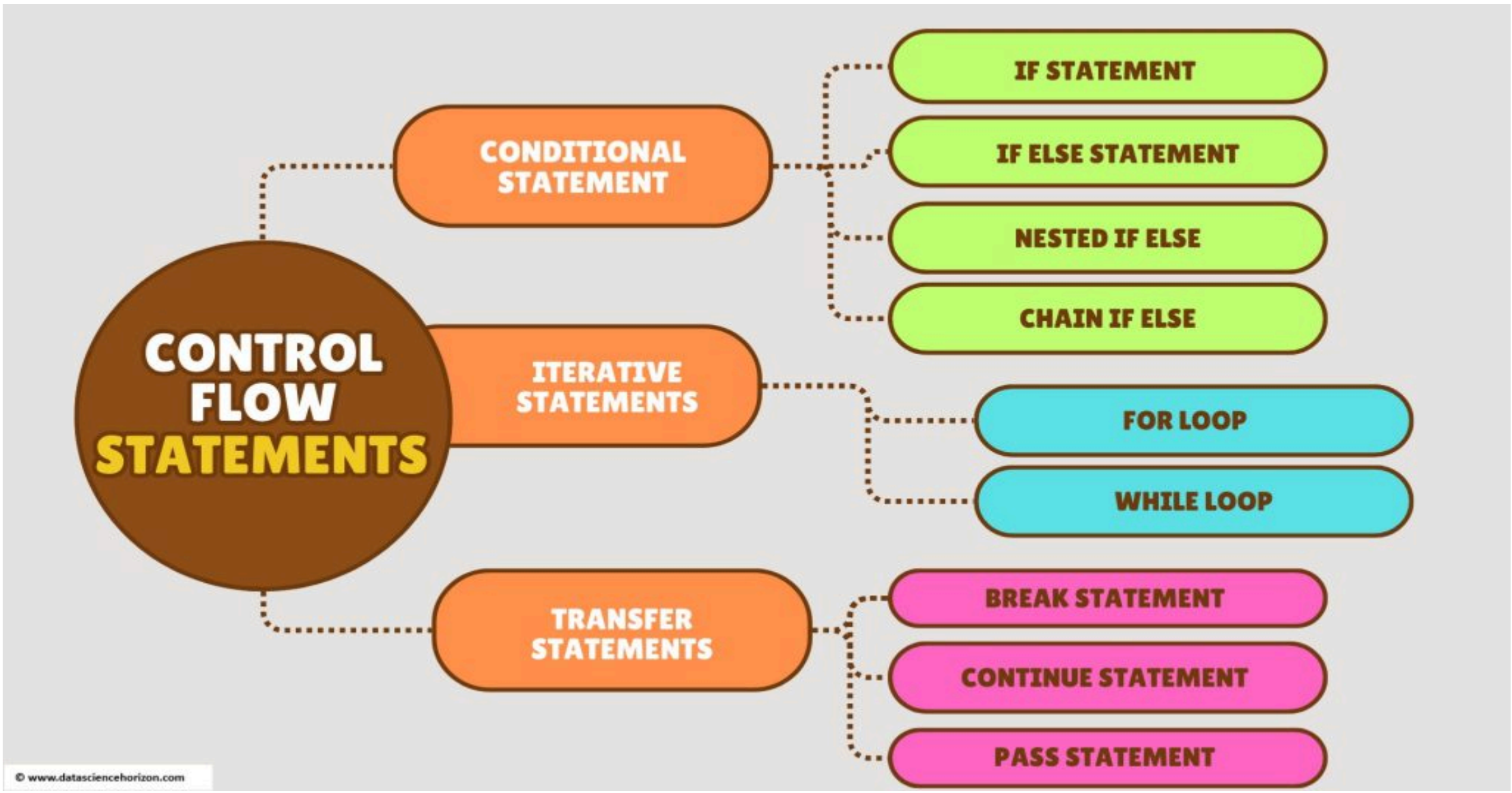
a -= 3       # Subtract and assign

a *= 2       # Multiply and assign

a /= 4       # Divide and assign

print(a)
```

🔀 Conditional Statements in Python



🔹 *What are Conditional Statements?*

Conditional statements allow a program to **make decisions** and execute code **based on conditions** (`True` or `False`).

🏠 *Real-life example*

If it rains ☔ → take umbrella
Else → go normally, Same logic in Python.

1️⃣ *if Statement*

Used when **only one condition** is checked.

```
age = 20
#✔️ Executes only if condition is True
if age >= 18:
    print("Eligible to vote")
```

---

2 *if–else Statement*

Used when there are **two choices**.

```
marks = 30

if marks >= 35:

    print("Pass")

else:

    print("Fail")
```

---

3 *if–elif–else Statement*

Used when there are **multiple conditions**.

```
marks = 85

if marks >= 90:

    print("Grade A")

elif marks >= 75:

    print("Grade B")

elif marks >= 60:

    print("Grade C")

else:

    print("Fail")

#OUTPUT: Grade B
```

---

4 *Nested if Statement*

if inside another if

```
age = 22

citizen = True

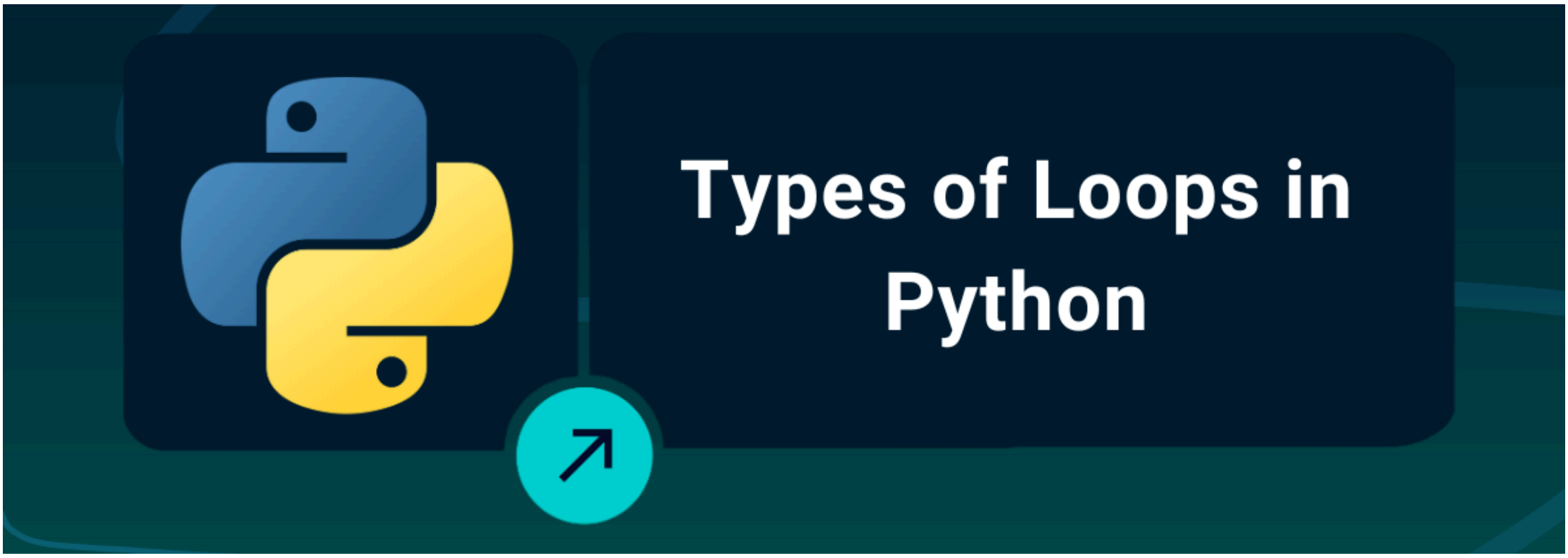
if age >= 18:

    if citizen:

        print("Eligible to vote")

#OUTPUT:Eligible to vote
```

🔁 *Looping Statements in Python (for, while)*



What are Loops?

Loops are used to **repeat a block of code multiple times** until a condition is satisfied.

🏠 *Real-Time Example*

Imagine you tell your assistant: “Print ‘Welcome’ 5 times.”

1️⃣ ***for Loop:*** Used when the **number of iterations is known**.

```
for i in range(1 , 5):

   print(i)

#output: 1 2 3 4
```

➤ **Example 2: Loop through string**

```
for ch in "Python":
   print(ch)
# P y t h o n
```

2️⃣ **while Loop:** Used when the **number of iterations is not known**.

➤ **Example 1: Print numbers**

```
i = 1

while i <= 5:

   print(i)

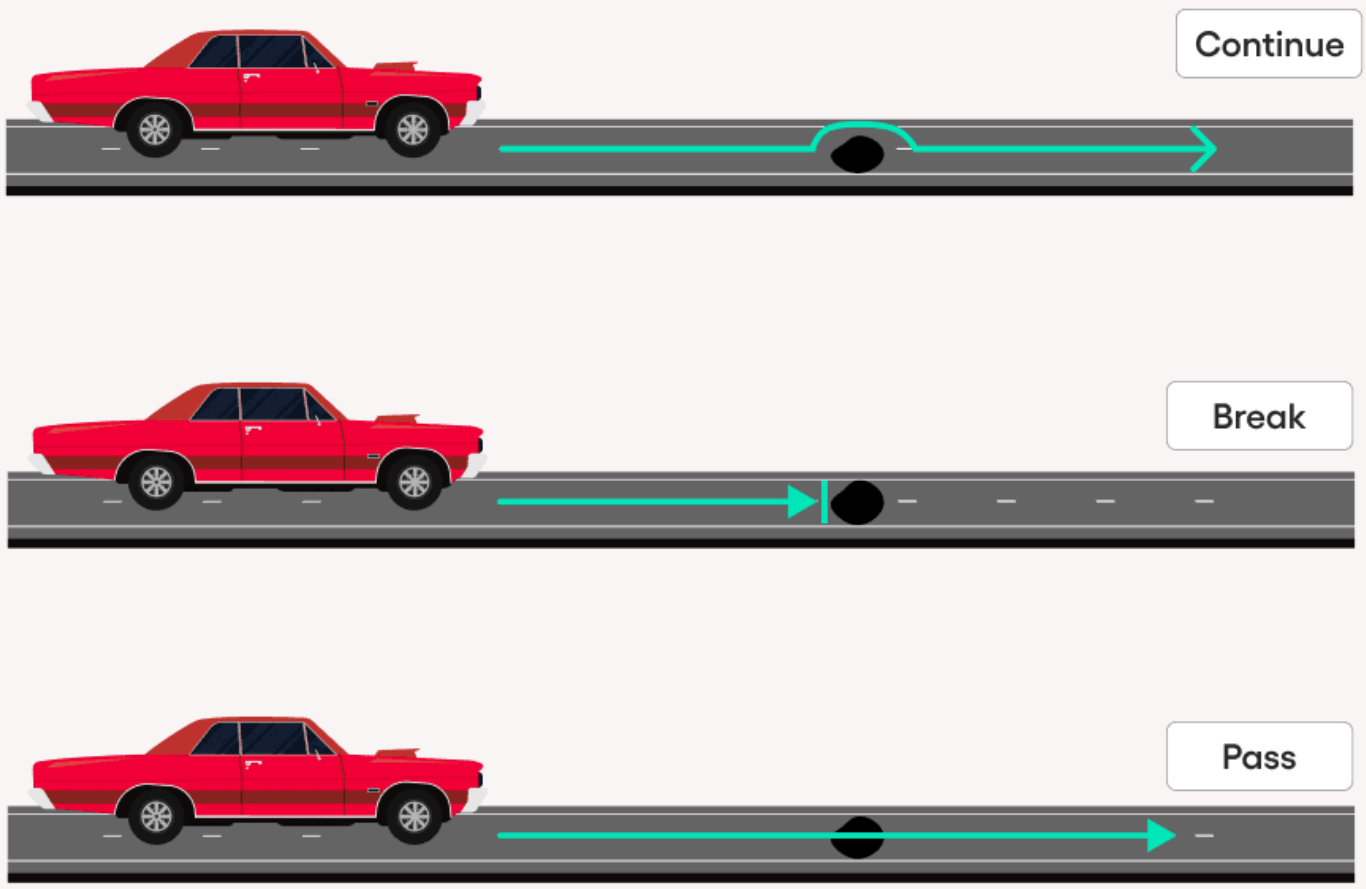
   i += 1 # 1 2 3 4 5
```

➤ *Example 2: Password check*

```
password = ""
while password != "admin":
    password = input("Enter password: ")
```

---

3 *Loop Control Statements*



➤ **break:** Stops the loop

```
for i in range(1, 6):
    if i == 3:
        break
    print(i) # 1 2
```

➤ **continue:** Skips current iteration

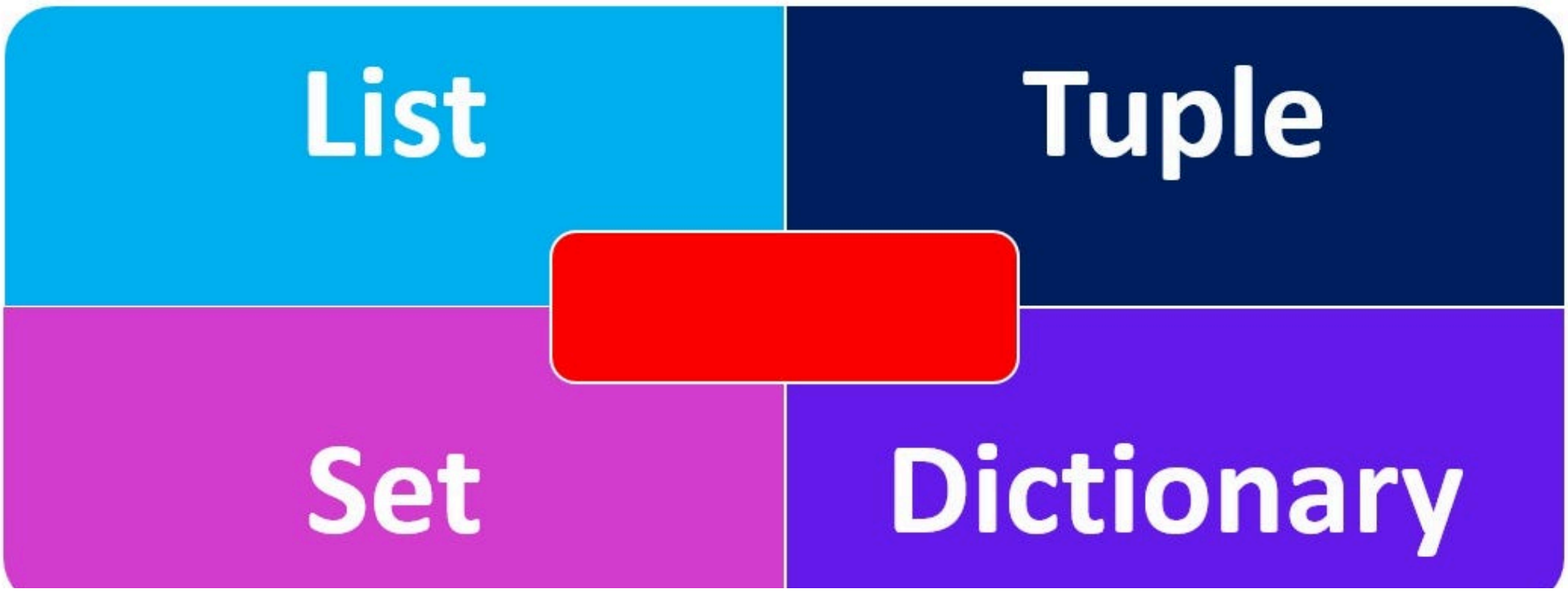
```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# output: 1 2 4 5
```

➤ **pass:** Does nothing (placeholder)

pass means: “Do nothing, just go ahead.”

```
for i in range(3):
    pass
```

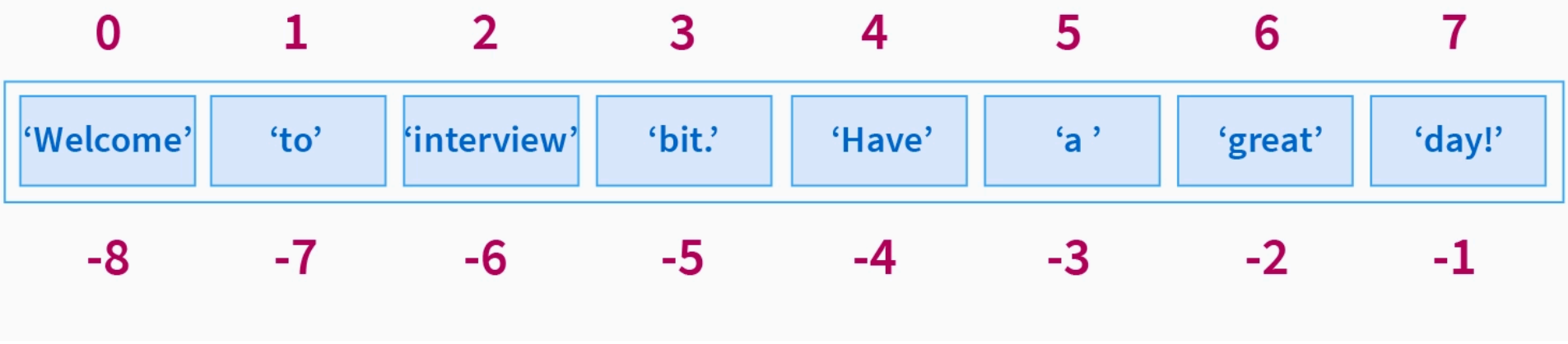
Collections in Python



📋 *List in Python*

A **List** in Python is a **collection of multiple values stored in a single variable.**

- 📑 Lists are **ordered**
- 🔄 Lists are **changeable (mutable)**
- ♻️ Lists allow **duplicate values**
- Written using **square brackets []**




```
List = ['welcome','to','interview','bit.','have','a','great','day.']

print(List[0]) #Output: welcome
```

1️⃣ **append()**: Adds one element at the end of the list.

```
students = ["Ravi", "Aman", "Neha"]

students.append("Mohan")

print(students)

# Output: ['Ravi', 'Aman', 'Neha', 'Mohan']
```

2 **insert()**: Adds an element at a specific index.

```
names = ["Ravi", "Aman", "Neha"]

names.insert(1, "Priya")

print(names) # Output: ['Ravi', 'Priya', 'Aman', 'Neha']
```

---

3 **remove()**: Removes a specific value from the list..

```
numbers = [10, 20, 30, 40]

numbers.remove(20)

print(numbers)

# Output: [10, 30, 40]
```

---

4 **pop()**: Removes an element using index (default removes last).

```
cart = ["Pen", "Book", "Bag"]

cart.pop()

print(cart) # Output:  ['Pen', 'Book']
```

---

5 **clear()**: Removes all elements from the list.

```
tasks = ["Task1", "Task2", "Task3"]

tasks.clear()

print(tasks)# Output: []
```

---

6 **count()**: When you want to check the frequency of an element.

```
numbers = [1, 5, 3, 5, 7, 5]

total = numbers.count(5)

print(total) # Output: 3
```

---

7 **sort()**: Sorts the list in ascending order (default).

```
numbers = [50, 10, 40, 20]

numbers.sort()

print(numbers) # Output:[10, 20, 40, 50]
```



📦 *Tuple in Python*

A **Tuple** in Python is a collection that is:

- Maintain the Ordered 📑
- Immutable (Cannot be changed) 🔒
- Allows duplicate values ♻️
- Tuple is written using **round brackets ( )**

🏠 *Real-time Example*

Think about your **date of birth**.
Example: (15, August, 2002) You cannot change it later. It stays fixed forever.

```python
# Tuple example

marks = (70, 80, 90)

print(marks)

print(type(marks)) # <class 'tuple'>
```

👨‍💻 *Code Example (Python)*

```python
numbers = (10, 20, 30, 40)

names = ("Mohan", "Rahul", "Asha")

mixed = (1, "Python", 3.5, True)

print(numbers)

print(names)

print(mixed)
```

1️⃣ **count():** Counts how many times a value appears in the tuple.

```
numbers = (1, 5, 3, 5, 7, 5)

total = numbers.count(5)

print(total)

# Output: 3
```

2️⃣ **index():** Returns the index position of a specific value.

```
numbers = (10, 20, 50, 40)

position = numbers.index(50)

print(position)

# Output: 2
```

⭐ *Important Notes About Tuple*

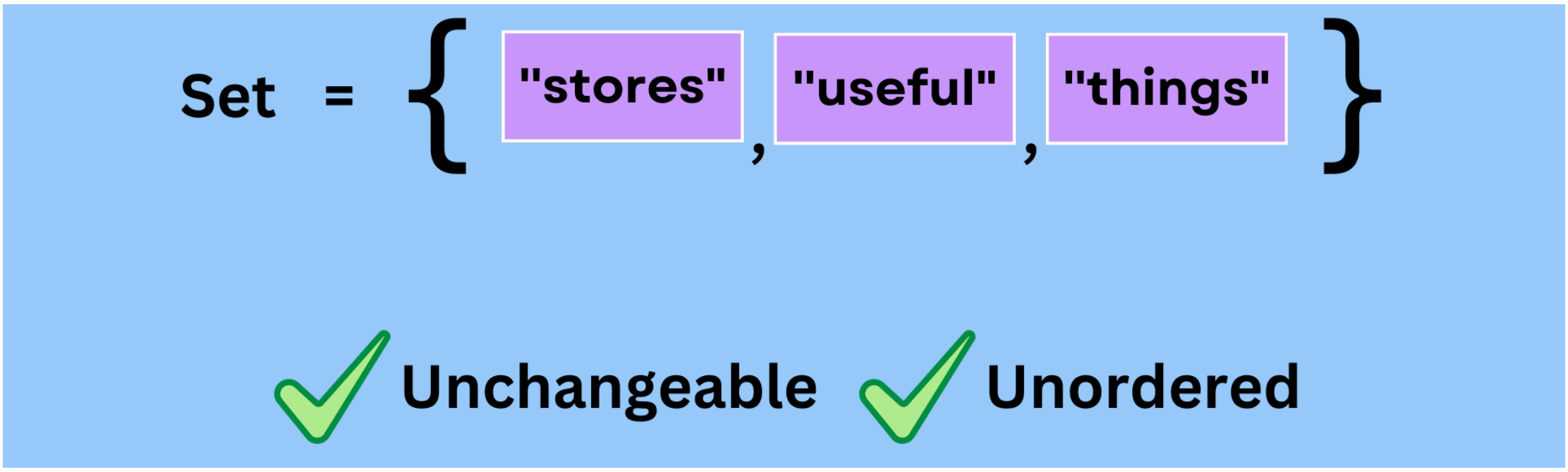
🚫 You cannot use:

- append()
- remove()
- insert()
- clear()
- sort()
- Because tuples are **immutable (cannot change after creation)**.

🆚 List vs Tuple (Quick Difference)

Feature	List	Tuple
Brackets	[ ]	( )
Changeable	Yes	No
Methods	Many	Only 2
Speed	Slower	Faster

Set in Python



Set in Python

A **Set** in Python is a collection that is:

- Unordered 🔄
- Unindexed 🚫
- No duplicate values ❌
- Mutable (can add/remove items) 🔧

Set is written using **curly brackets { }**

👨‍💻 Code Example (Python)

```
numbers = {1, 2, 3, 4}

names = {"Mohan", "Rahul", "Asha"}

mixed = {1, "Python", 3.5}

print(numbers)

print(names)
```

---

1 add(): Adds one element to the set.

```
numbers = {10, 20, 30}

numbers.add(50)

print(numbers)

# Output:  {10, 20, 30, 50}
```

---

2 update(): Adds multiple elements to the set. When adding more than one value.

```
numbers = {1, 2, 3}

numbers.update([4, 5, 6])
```

```
print(numbers)

# Output:  {1, 2, 3, 4, 5, 6}
```

3️⃣ **remove():** Removes a specific element.

```
numbers = {10, 20, 30}

numbers.remove(20)

print(numbers)

# Output: {10, 30}
```

4️⃣ **discard():** Removes an element (no error if not found). Safe removal.

```
numbers = {10, 20, 30}

numbers.discard(100)

print(numbers)

# Output: {10, 20, 30}
```

5️⃣ **pop():** Removes a random element.

```
numbers = {10, 20, 30}

numbers.pop()

print(numbers)

# Output:  One element will be removed (random)
```

6️⃣ **clear():** Removes all elements from the set. To make the set empty.

```
numbers = {1, 2, 3}

numbers.clear()

print(numbers)

# Output:  set()
```

---

7 **union()**: Combines two sets (no duplicates). To merge sets.

```
set1 = {1, 2, 3}

set2 = {3, 4, 5}

result = set1.union(set2)

print(result) # Output: {1, 2, 3, 4, 5}
```

---

8 **intersection()**: Returns common elements. To find matching values.

```
set1 = {1, 2, 3}

set2 = {2, 3, 4}

result = set1.intersection(set2)

print(result) # Output:  {2, 3}
```

---

9 **difference()**: Returns elements present in the first set but not in the second. To find unique values.

```
set1 = {1, 2, 3}

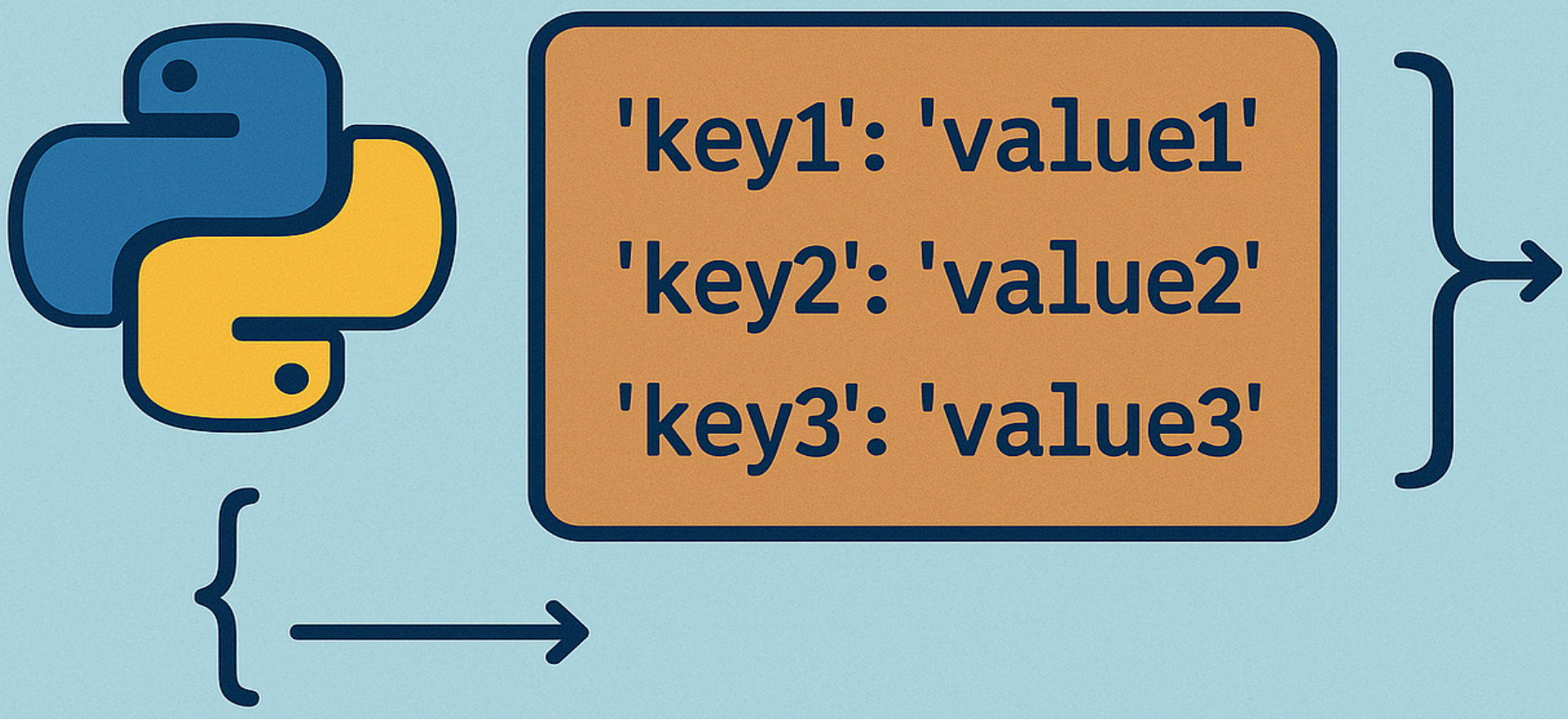
set2 = {2, 3, 4}

result = set1.difference(set2)

print(result)

# Output:  {1}
```

PYTHON DICTIONARY



🔹 *What is a Dictionary?*

- A **Dictionary** stores data in **key and value** form.
- Each key has one value.
- It is written using curly brackets { }.
- Keys must be unique.

🏠 *Real-time Example*

Think about a **phone contact list**. Name is the key. The phone number is the value. Each name has its own number.

🔥 *Real-Life Example*

```
employee = {
    "id": 1001,
    "name": "Ravi",
    "salary": 50000,
    "department": "IT"}
print(employee)
# Output: {'id': 1001, 'name': 'Ravi', 'salary': 50000, 'department': 'IT'}
```

1 **get():** When you want to get a value without error if the key is not found.

```
student = {"name": "Mohan", "age": 22}

value = student.get("name")

print(value) # Output: Mohan
```

---

2 **keys():** Returns all keys in the dictionary. When you want to see all the keys.

```
student = {"name": "Mohan", "age": 22}

print(student.keys())

# Output:  dict_keys(['name', 'age'])
```

---

3 **values():** Returns all values.

```
student = {"name": "Mohan", "age": 22}

print(student.values())

# Output:  dict_values(['Mohan', 22])
```

---

4 **items():** Returns key-value pairs.

```
student = {"name": "Mohan", "age": 22}

print(student.items())

# Output: dict_items([('name', 'Mohan'), ('age', 22)])
```

---

5 **update():** Adds or updates key-value pairs. When modifying a dictionary.

```
student = {"name": "Mohan", "age": 22}

student.update({"age": 23})

print(student) # Output: {'name': 'Mohan', 'age': 23}
```

---

6 **pop():** Removes a key and returns its value. When deleting a specific key.

```
student = {"name": "Mohan", "age": 22}

student.pop("age")

print(student) # Output: {'name': 'Mohan'}
```
